

Lesson 8: Game Logic, Timers, and Debugging Like a Pro

Mission Brief

Mission Control has one final systems check before graduation: the Explorer needs to build interactive *games*, not just forms and lists. Games add three new ingredients — randomness, timing, and multi-step state comparisons — and they surface bugs that never show up in a simple to-do app. Today you learn to shuffle data fairly, control time itself with `setTimeout` / `setInterval`, compare state across multiple user actions, and use DevTools like a real engineer to squash the bugs that inevitably follow.

Learning Objectives

By the end of this lesson, you will be able to:

- Shuffle an array of items into random order using the Fisher-Yates algorithm
- Use `setTimeout()` to delay an action and `clearTimeout()` to cancel it
- Track and compare two or more "selected" items across separate click events
- Lock user input temporarily while an animation or check is in progress
- Use `console.log`, breakpoints, and the DevTools Sources panel to trace a bug to its root cause

What You'll Build

A "Signal Match" mini-game: eight symbol tiles (four pairs) are shuffled and shown face-down. Clicking a tile flips it face-up; clicking a second tile checks if they match. Matching pairs stay revealed permanently; non-matching pairs flip back after a short delay. A move counter and a "you win" message complete the loop — the exact core logic Project 06's full Memory Card Game is built from.

Prerequisites

- Lesson 2 (DOM manipulation)
- Lesson 3 (event listeners)
- Comfort with arrays, `.map()`, and object shorthand

Key Concepts

- The Fisher-Yates shuffle algorithm
- `setTimeout()` / `clearTimeout()` — scheduling and cancelling delayed code
- Tracking "first pick" / "second pick" state across two separate clicks

- Temporarily disabling input with a boolean "lock" flag
- Debugging with `console.log` , `debugger` , and DevTools breakpoints

Concept Explanation

Randomly shuffling an array sounds simple, but a naive approach (like sorting with `Math.random()` as the comparator) produces **biased** results — some orderings come up more often than others. The standard, provably fair algorithm is the **Fisher-Yates shuffle**: walk the array backwards from the last index, and at each step swap the current element with a random element at or before its position. This guarantees every possible ordering is equally likely, and it runs in a single pass with no extra memory.

Games constantly need to **delay** things: flip a wrong pair back over after the player has a moment to see it, show a "Game Over" message a beat after the last move, and so on. `setTimeout(callback, milliseconds)` schedules `callback` to run once, after that delay, without freezing the rest of the page — the browser keeps handling clicks and rendering while it waits. It returns a numeric ID that can be passed to `clearTimeout(id)` to cancel it before it fires, which matters if, say, the player somehow triggers a new action before the old delayed action was due to run. `setInterval()` is the repeating cousin, used for things like countdown timers, and is cancelled with `clearInterval()` .

Memory-style games require comparing **two separate clicks** that happen at different times, which means you need state variables that persist *between* click events — not just inside one handler. A clean pattern: keep a `selectedTiles` array that starts empty; the first click pushes a tile into it and flips it face-up; the second click pushes the second tile, flips it up, and *immediately* checks whether the two tiles match, updates the score/board accordingly, and resets `selectedTiles` back to empty for the next pair.

A critical detail games teach that simpler apps don't: you often need to **temporarily disable input**. While two mismatched tiles are being shown before flipping back, a fast player could click a third tile and corrupt your comparison logic — so a simple boolean flag like `isChecking = true` (set right before the delay, cleared right after) is enough to make every click handler bail out early with a guard clause until it's safe again.

Finally, this is the natural moment to formalize **debugging technique**, since game logic bugs are often invisible from just reading the code. `console.log()` at key checkpoints (which tile was clicked, what's in `selectedTiles` , whether a match was found) is still your fastest first tool. When that's not enough, DevTools' **Sources panel** lets you set a real **breakpoint** — click a line number to pause execution exactly there — and step through your code line by line, inspecting every variable's actual live value, which is far more precise than guessing where to place another `console.log` .

Guided Coding

```

<!-- index.html -->
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <title>Signal Match</title>
  <style>
    body { font-family: sans-serif; background: #0b0e1a; color: #e6ecff; padding: 2rem; text-align: center; }
    #board { display: grid; grid-template-columns: repeat(4, 70px); gap: 10px; justify-content: center; margin: 1.5rem auto; }
    .tile { width: 70px; height: 70px; background: #1e2540; border-radius: 8px; display: flex; align-items: center; justify-content: center; font-size: 1.8rem; cursor: pointer; user-select: none; }
    .tile.revealed { background: #2f3a63; }
    .tile.matched { background: #1c3a2c; cursor: default; }
    #status { min-height: 1.5em; color: #9fd8ff; }
  </style>
</head>
<body>
  <h1>Signal Match</h1>
  <p>Moves: <span id="move-count">0</span></p>
  <div id="board"></div>
  <p id="status"></p>
  <button id="restart-btn">Restart</button>
  <script src="script.js"></script>
</body>
</html>

```

```

// script.js

const SYMBOLS = ["🐙", "🐙", "🚀", "★"];
const boardEl = document.querySelector("#board");
const moveCountEl = document.querySelector("#move-count");
const statusEl = document.querySelector("#status");
const restartBtn = document.querySelector("#restart-btn");

let tiles = []; // array of { id, symbol, isRevealed, isMatched }
let selectedIds = []; // ids of tile(s) currently flipped, awaiting a match check
let isChecking = false; // input lock while a mismatch is being shown
let moveCount = 0;

// --- FISHER-YATES SHUFFLE ---
function shuffle(array) {
  const result = [...array]; // never mutate the original array
  for (let i = result.length - 1; i > 0; i--) {
    const j = Math.floor(Math.random() * (i + 1));
    [result[i], result[j]] = [result[j], result[i]]; // swap
  }
  return result;
}

// --- BUILD A FRESH SET OF TILES (each symbol appears exactly twice) ---
function createTiles() {
  const doubledSymbols = shuffle([...SYMBOLS, ...SYMBOLS]);
  return doubledSymbols.map((symbol, index) => ({
    id: index,
    symbol: symbol,
    isRevealed: false,
    isMatched: false,
  }));
}

// --- RENDER (rebuild the board from current state) ---
function render() {
  boardEl.innerHTML = "";
  tiles.forEach((tile) => {
    const tileEl = document.createElement("div");
    tileEl.classList.add("tile");
    if (tile.isMatched) tileEl.classList.add("matched");
    else if (tile.isRevealed) tileEl.classList.add("revealed");
    tileEl.textContent = tile.isRevealed || tile.isMatched ? tile.symbol : "";
    tileEl.dataset.id = tile.id;
    boardEl.appendChild(tileEl);
  });
  moveCountEl.textContent = moveCount;
}

```

```

// --- CLICK HANDLING (event delegation) ---
boardEl.addEventListener("click", (event) => {
  const tileEl = event.target.closest(".tile");
  if (!tileEl || isChecking) return; // guard clause: ignore clicks while locked or off-board

  const id = Number(tileEl.dataset.id);
  const tile = tiles.find((t) => t.id === id);

  // Ignore clicks on an already-revealed, already-matched, or already-selected tile
  if (!tile || tile.isMatched || tile.isRevealed || selectedIds.includes(id)) return;

  tile.isRevealed = true;
  selectedIds.push(id);
  render();

  if (selectedIds.length === 2) {
    moveCount++;
    checkForMatch();
  }
});

function checkForMatch() {
  const [firstId, secondId] = selectedIds;
  const firstTile = tiles.find((t) => t.id === firstId);
  const secondTile = tiles.find((t) => t.id === secondId);

  if (firstTile.symbol === secondTile.symbol) {
    firstTile.isMatched = true;
    secondTile.isMatched = true;
    selectedIds = [];
    render();
    checkForWin();
  } else {
    isChecking = true; // lock input so the player can't click a third tile mid-check
    setTimeout(() => {
      firstTile.isRevealed = false;
      secondTile.isRevealed = false;
      selectedIds = [];
      isChecking = false;
      render();
    }, 800);
  }
}

function checkForWin() {
  const allMatched = tiles.every((tile) => tile.isMatched);
  statusEl.textContent = allMatched ? `You matched every signal in ${moveCount} moves!` : "";
}

```

```
function startNewGame() {
  tiles = createTiles();
  selectedIds = [];
  isChecking = false;
  moveCount = 0;
  statusEl.textContent = "";
  render();
}

restartBtn.addEventListener("click", startNewGame);

// --- START ---
startNewGame();
```

Code Walkthrough

- **shuffle()**: implements Fisher-Yates by walking the array from the end backwards (`i` from `length - 1` down to `1`), picking a random index `j` from `0` to `i` inclusive, and swapping. Copying the array first (`[...array]`) means the original `SYMBOLS` list is never mutated, which matters if `createTiles()` is ever called again.
- **createTiles()**: doubles the four symbols into eight, shuffles them once, and maps each into a full tile object — this is the app's entire data model, mirroring the `{ id, text, completed }` pattern from Lesson 7's to-do app.
- **selectedIds / isChecking**: the two pieces of state that only make sense *because* this is a two-click game — `selectedIds` remembers what was picked first while waiting for a second click, and `isChecking` is the guard flag that prevents a third click from interfering while a mismatch is still being displayed.
- **The click handler's guard clauses**: notice how many `return` statements come before any real logic — ignoring clicks while locked, on the board's empty space, on an already-matched tile, an already-revealed tile, or a tile already in `selectedIds`. Each guard clause removes one entire category of possible bug.
- **checkForMatch()**: on a match, it updates state and re-renders immediately (no delay needed — showing the match instantly feels correct). On a mismatch, it sets `isChecking = true` *before* scheduling `setTimeout`, so no clicks land as "wrong" input during the 800ms delay, and only turns it back to `false` inside the delayed callback itself.
- **checkForWin()**: uses `.every()`, the mirror method to `.some()`, which returns `true` only if *every* element passes the test — a clean one-line way to detect "the whole board is matched."

Student Mission

1. Add a "best score" (fewest moves to win) saved to Local Storage using Lesson 6's pattern, displayed and updated only when a new game beats the previous best.

2. Add a live timer (seconds elapsed) using `setInterval()`, started on the first tile click and stopped with `clearInterval()` the moment the player wins.
3. Increase the board to 6 pairs (12 tiles) and adjust the CSS grid to fit, confirming the shuffle and matching logic still work correctly with the larger array.

Challenge

Add a simple three-level difficulty selector (Easy = 4 pairs, Medium = 6 pairs, Hard = 8 pairs) that rebuilds `SYMBOLS` ' active subset and calls `startNewGame()` fresh for the chosen size — practicing driving your data model from a user choice rather than a hardcoded constant.

Experiment/Extension

Open DevTools' **Sources** panel, find your `script.js`, and click the line number inside `checkForMatch()` where `firstTile.symbol === secondTile.symbol` is evaluated to set a real breakpoint. Play the game until execution pauses there, then hover over `firstTile` and `secondTile` in the paused code to inspect their actual live values, and use the "step over" control to advance line by line — this is strictly more reliable than sprinkling `console.log` everywhere, and it's how professional developers actually hunt real bugs.

Common Mistakes

- Shuffling with `array.sort(() => Math.random() - 0.5)` — it looks correct but is statistically biased, producing certain orders more often than others; always use Fisher-Yates for real randomness.
- Forgetting the `isChecking` lock, which lets a fast player click a third tile mid-check and silently corrupts `selectedIds` into holding more than two ids.
- Comparing tiles by array index instead of by their stable `id`, which breaks the moment tiles are ever removed, reordered, or the render rebuilds the DOM.
- Calling `render()` before updating `moveCount` / `isChecking` /state, so the screen briefly shows stale information one step behind reality.
- Not clearing `selectedIds` back to `[]` in *both* the match branch and the mismatch branch — forgetting it in one path silently breaks every game after the first pair.

Debugging Tips

- `console.log(selectedIds, isChecking)` at the very top of the board's click handler is the single fastest way to see why a click is being ignored (or wrongly accepted).
- If tiles flip back instantly instead of after a delay, confirm `setTimeout` 's second argument is actually a number of milliseconds (800), not accidentally omitted or set to 0.
- If the game "locks up" after a mismatch and never lets you click again, you likely forgot to set `isChecking = false` inside the `setTimeout` callback itself.

- Use the DevTools breakpoint technique from the Experiment section any time `console.log` output alone doesn't explain what's happening — pausing execution and inspecting real values beats guessing every time.

Check Your Understanding

1. Why does sorting an array with a random comparator function produce biased shuffles, while Fisher-Yates does not?
2. What does `setTimeout()` return, and what is that value used for?
3. Why is a boolean "lock" flag like `isChecking` necessary in this game but wasn't needed in Lesson 7's to-do app?
4. What does `.every()` return, and how does `checkForWin()` use it?
5. What is one advantage a DevTools breakpoint has over a `console.log` statement?

Mini Quiz + Answer Key

Q1. What algorithm does `shuffle()` implement? A) Bubble sort B) Fisher-Yates shuffle C) Binary search D) Quicksort

Q2. What does `clearTimeout(id)` do? A) Runs the scheduled callback immediately B) Cancels a scheduled `setTimeout` before it fires C) Deletes all timers on the page D) Pauses the whole script

Q3. In this game, what is `selectedIds` used for? A) Storing every tile that has ever been clicked B) Tracking the one or two tiles currently flipped, awaiting a match check C) Storing the winning symbols only D) It is unused

Q4. Why is `isChecking` set to `true` before calling `setTimeout` in the mismatch branch? A) It has no real purpose B) To lock input so a third click can't corrupt the comparison while the mismatch is still visible C) To make the delay run faster D) It is required syntax for `setTimeout`

Answer Key: 1-B, 2-B, 3-B, 4-B

Lesson Recap

You learned to shuffle data fairly with Fisher-Yates, schedule and cancel delayed actions with `setTimeout` / `clearTimeout`, track state across multiple separate click events, lock input temporarily with a boolean flag, and use DevTools breakpoints to debug logic that `console.log` alone can't easily explain — completing the full toolkit this course set out to teach and directly preparing you for Project 06, the Memory Card Game.

Homework

Add a "shuffle penalty": if the player clicks two mismatched tiles more than 3 times in a row, briefly shuffle the positions of all still-hidden tiles (using your existing `shuffle()` function) before letting them continue, practicing calling your own utility function from a brand-new rule you design yourself.

Portfolio Project Connection

This lesson is the direct blueprint for **Project 06, the Memory Card Game** — the shuffle algorithm, two-click comparison state, `setTimeout`-based flip-back delay, and win detection built here are exactly the mechanics the full portfolio project expands into a complete, polished game.